

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"

/* Private includes ----- */
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */

/* Private typedef ----- */
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ----- */
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro ----- */
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ----- */
TIM_HandleTypeDef htim2;

/* USER CODE BEGIN PV */
uint8_t sayac = 0;
uint8_t hedef_bayrak = 0;
/* USER CODE END PV */

/* Private function prototypes ----- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ----- */
/* USER CODE BEGIN 0 */
void lcd_send_half(uint8_t data) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, (data & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, (data & 0x02) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, (data & 0x04) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, (data & 0x08) ? GPIO_PIN_SET : GPIO_PIN_RESET);

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
    for(volatile int i=0; i<1000; i++);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET);
    for(volatile int i=0; i<1000; i++);
}

void lcd_cmd(uint8_t cmd) {

```

```

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
    lcd_send_half(cmd >> 4);
    lcd_send_half(cmd);
}

void lcd_data(uint8_t data) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET);
    lcd_send_half(data >> 4);
    lcd_send_half(data);
}

void lcd_init(void) {
    HAL_Delay(50);
    lcd_send_half(0x03); HAL_Delay(5);
    lcd_send_half(0x03); HAL_Delay(1);
    lcd_send_half(0x03); HAL_Delay(1);
    lcd_send_half(0x02); HAL_Delay(1);

    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x01);
    HAL_Delay(2);
}

void lcd_set_cursor(uint8_t row, uint8_t col) {
    uint8_t addr = (row == 0) ? 0x80 : 0xC0;
    lcd_cmd(addr + col);
}

void lcd_send_string(char *str) {
    while (*str) lcd_data(*str++);
}
/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();

```

```

MX_TIM2_Init();

/* USER CODE BEGIN 2 */

// 1. İşlemcimiz Proteus kilitlenmesin diye 8 MHz'de çalışıyor.
// 8 MHz saati 1 mikrosaniyeye bölmek için Prescaler (PSC) 7 yapıldı (8/8 = 1)
TIM2->PSC = 7;
TIM2->ARR = 249;

// 2. Timer 2 Donanımsal Kesmesini (NVIC) Manuel Aktif Ediyoruz
HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(TIM2_IRQn);

// 3. LCD Başlatma
lcd_init();

// 4. Ödev Şartı: Açılış Ekranı (3 Saniye)
lcd_set_cursor(0, 2);
lcd_send_string("Eyupcan Cil");
lcd_set_cursor(1, 0);
lcd_send_string("No: 23010104094");
HAL_Delay(3000); // Gerçek 8 MHz saat hızıyla Proteus'ta tam 3 saniye bekler

// 5. Ödev Şartı: Ana Ekran
lcd_cmd(0x01);
HAL_Delay(2);

// Üst satır ortalanmış
lcd_set_cursor(0, 4);
lcd_send_string("SUBU EEM");

// Alt satır ortalanmış (Sayma başlamadan önceki 00 anını hemen ekrana bas)
char buffer[16];
sprintf(buffer, "SAYAC: %02d", sayac);
lcd_set_cursor(1, 3);
lcd_send_string(buffer);

// 6. Timer Kesmesini Başlat
HAL_TIM_Base_Start_IT(&htim2);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    if (hedef_bayrak == 1)
    {
        hedef_bayrak = 0;
        char buffer[16];
        sprintf(buffer, "SAYAC: %02d", sayac);
        lcd_set_cursor(1, 3);
        lcd_send_string(buffer);
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**

```

```

* @brief System Clock Configuration
* @retval None
*/
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    // Proteus dostu, kilitlenmeyen saf 8 MHz Dahili Osilatör (HSI) ayarı
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
* @brief TIM2 Initialization Function
* @param None
* @retval None
*/
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 16;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 65535;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */

/* USER CODE END TIM2_Init 2 */
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4
                      |GPIO_PIN_5|GPIO_PIN_6, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);

    /*Configure GPIO pins : PA1 PA2 PA3 PA4
                           PA5 PA6 */
    GPIO_InitStruct.Pin = GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4
                      |GPIO_PIN_5|GPIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /*Configure GPIO pin : PB0 */
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {

```

```

// Kesme periyodunu (250 us) osiloskopta görmek için pini toggle yap
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);

static uint16_t kesme_sayici = 0;
kesme_sayici++;

// 250 us * 2000 = 500,000 us = 500 ms hedefine ulaşma kontrolü
if (kesme_sayici >= 2000)
{
    kesme_sayici = 0;
    sayac++;

    if (sayac > 99)
    {
        sayac = 0;
    }

    hedef_bayrak = 1;
}
}
}
/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```